

Pseudoneighbor Algorithm

Ethan Custodio

We start with a full infinite length tangle W , where W^U and W^S are the unstable and stable manifolds, respectively. $W^U(\mathbf{x}, \mathbf{x}')$ is the open interval of W^U between the two points $\mathbf{x}, \mathbf{x}' \in W^U$. Two intersection points $\mathbf{x}, \mathbf{x}'$ are a *pseudoneighbor pair* if they satisfy

$$W^S(\mathbf{x}, \mathbf{x}') \cap X = W^U(\mathbf{x}, \mathbf{x}') \cap X = \emptyset.$$

Where,

$$X = \bigcup_{n \in \mathbb{Z}} \left\{ \mathcal{M}^n(\mathbf{x}) \mid \mathbf{x} \in T^S \cap T^U \right\}.$$

In other words, the open stable and unstable intervals connecting the pseudoneighbor pairs must contain no trellis intersections, nor any iterate of a trellis intersection.

For a finite trellis, all pseudoneighbor pairs will first appear on $W^S(r_n, r_{n+p})$ where r_n is the intersection with the greatest stable canonical distance and $r_{n+p} = \mathcal{M}^p(r_n)$. Here p is the period of the stable branch. It is sufficient to check only this interval to discover all pseudoneighbor pairs. Then take the intersections from that interval and map them forward and backward to generate the entire pseudoneighbor pair trajectory.

For each pair of pseudoneighbors we punch a hole in the region bounded by the pair. This hole is placed infinitesimally close to one of the two pseudoneighbors.

Algorithm 1 COMPUTE-PSEUDONEIGHBORS(T) - Returns the set of all pseudoneighbors

```

1: Let  $N$  be the set of all intersections in the trellis that have been computed
2: Let  $P$  be the set of all pseudoneighbors that have been found. Initially set  $P = \emptyset$ 
   * first from each brnach orbit *
3: for each stable branch  $W_{z_i}^s$  do
4:   Let  $x_0 \leftarrow r_n$  where  $r_n \in N$ 
5:   Let  $x_1 \leftarrow \text{next-intersection}(x_0)$ 
6:   while  $x_0$  is not  $r_{n+p}$  do
7:     if all intersections in  $N$  are not on and do not map to  $W_{z_i}^U(x_0, x_1)$  then
8:       add  $(x_0, x_1)$  to  $P$ 
       * include how we will use the points in  $N$  to determine if they are on that interval *
9:     end if
10:     $x_0 \leftarrow x_1$ 
11:     $x_1 \leftarrow \text{next-intersection}(x_1)$ 
12:   end while
13: end for
14: return  $P$ 
```

Note: Here next-intersection is a function which returns the next intersection on the stable manifold walking towards the anchor point.
